WasteLess: AI Demand Forecasting to Cut Restaurant Food Waste

Oct 9, 2026 · @Himanshu

1. Project overview

WasteLess is a multi-tenant web platform that predicts how much of each dish a restaurant will sell, then turns that prediction into a daily prep and purchase plan. It is built with Next.js for the product and a Python machine learning service for the intelligence.

Problem statement

Small and mid-sized food businesses decide how much to cook by guesswork. Cooking too much creates waste and lost money. Cooking too little means lost sales and unhappy customers.

Large chains use expensive forecasting systems. Cafés, college canteens, cloud kitchens and family restaurants rarely have access to anything similar.

Proposed solution

WasteLess learns from each outlet's past sales, plus weather, holidays and local events. It forecasts demand per dish, converts it into ingredient quantities using recipes, and tracks what was actually sold and wasted. Every day of logged data makes the next forecast more accurate.

Objectives

1. Forecast dish-level demand with lower error than a naive "same day last week" baseline.
2. Turn forecasts into concrete prep lists and purchase orders that staff can act on.
3. Measure and visualise waste, cost saved and CO_2 avoided.
4. Explain every forecast in plain language so owners trust it.
5. Deliver a complete, deployable product with real roles and workflows.

Scope

In scope: web app (desktop and tablet), forecasting service, waste tracking, inventory, analytics, AI assistant.

Out of scope: native mobile apps, payments and billing, deep integration with commercial POS systems (a mock POS API is used instead).

2. Target users and roles

WasteLess serves four roles, each with its own permissions and screens.

Role	Who they are	What they can do
Owner	Business owner, may run several outlets	Sees all outlets, analytics and reports; manages staff and settings
Manager	Outlet or kitchen manager	Manages menu, recipes, inventory and suppliers; approves daily prep plans
Kitchen staff	Cooks and helpers	Views the prep list; logs production, sales and waste in kitchen mode
Platform admin	The WasteLess team	Manages all tenants, monitors usage and model health

Target customers include independent restaurants, cafés, cloud kitchens, college and office canteens, and hostel messes.

3. How the system works

After a one-time setup, WasteLess runs as a daily loop: forecast, plan, cook, log, learn.



daily workflow · 1 setup step, 6 repeating steps

Logged sales and waste flow back into the model, so forecasts improve the longer an outlet uses the system. Retraining itself runs weekly; the logging that feeds it happens every day.

A day in the life of an outlet:

1. **Overnight:** the scheduler fetches tomorrow's weather and holidays and generates forecasts for every dish.
2. **Morning:** the manager opens the prep list, adjusts a few numbers if needed, and approves it. Purchase orders for low stock are suggested.
3. **Service:** kitchen staff cook from the approved list and log sales per shift in kitchen mode.
4. **Closing:** staff photograph and log leftovers; the image model suggests what and how much was wasted.
5. **Anytime:** the owner checks the dashboard, asks the AI assistant questions, or runs what-if scenarios.

4. Feature modules

The system is split into 14 modules. Modules E, H, I, K and L contain machine learning; the rest form the product around them.

Module A: Authentication and multi-tenancy

Purpose: Let many businesses use one platform safely, with each user seeing only their own data.

- Sign up, login, email verification and password reset.
- Hierarchy of organisation, outlets and users, so one owner can run several branches.
- Role-based access control enforced on every page and API call.
- Staff invitations by email with a role assigned.

Module B: Data ingestion

Purpose: Get historical and daily sales into the system, cleanly.

- CSV or Excel upload of past sales with a column-mapping wizard (date, dish, quantity, price).
- Validation that flags missing days, duplicate rows and extreme values before import.
- Manual daily sales entry for outlets without a POS system.
- A mock POS API so sales can be pushed in automatically, simulating a real integration.

Module C: Menu and recipe management

Purpose: Describe what the outlet sells and what each dish is made of.

- Dishes with category, price, veg or non-veg tag, and active status.
- Recipe builder listing ingredients and quantity per plate.
- Automatic cost per plate and profit margin from current ingredient prices.

Module D: Inventory and suppliers

Purpose: Know what is in stock and what needs ordering.

- Ingredient stock with unit, shelf life and storage type.
- Stock-in and stock-out logs, with expiry tracking and "use first" highlights.
- Supplier directory with prices and delivery lead times.
- Purchase orders generated automatically from forecasts minus current stock.

Module E: Demand forecasting engine (ML)

Purpose: Predict how many plates of each dish will sell in the next 1 to 7 days.

- Forecasts per dish per day, generated every night.
- A range for each forecast (low, likely, high), not a single number.
- Uses weather, holidays, weekends, paydays and local events as inputs.

- Retrains weekly on new data; picks the best model per outlet.

Module F: Prep planning

Purpose: Turn forecasts into instructions the kitchen can follow.

- Daily prep list in plates and ingredient quantities, using recipes.
- Risk-aware quantities: closer to "likely" for costly dishes, closer to "high" for cheap sides.
- Managers can adjust and approve the plan; every adjustment is logged.
- Printable kitchen sheet and a tablet view.

Module G: Kitchen mode and daily operations

Purpose: Make data entry fast for busy staff.

- Large-button interface for logging prepared, sold and wasted quantities.
- Shift-wise logging, such as lunch and dinner.
- End-of-day closing checklist that locks the day's data.

Module H: Waste tracking with computer vision (ML)

Purpose: Measure waste with minimal effort from staff.

- Staff photograph leftover trays or the waste bin.
- An image model identifies the food items and estimates quantity.
- Staff confirm or correct the result before it is saved.
- Waste is tagged by reason: overproduction, spoilage, plate waste or preparation loss.

Module I: Analytics and insights (ML)

Purpose: Show owners what is happening and why.

- Dashboard of sales, forecast accuracy, waste, money saved and CO_2 avoided.
- Waste heatmap by dish and day of the week.
- Plain-language forecast explanations, for example "Higher than usual because it is Friday and rain is expected."
- Menu engineering that groups dishes by popularity and margin, with suggestions such as removing a low performer.

Module J: What-if simulator

Purpose: Let owners test decisions before making them.

- Change conditions such as rain, a holiday, a discount or a new dish, and see forecasts update.
- Compare predicted cost and waste of scenarios side by side.

Module K: Anomaly detection and alerts (ML)

Purpose: Catch problems early.

- Flags unusual sales spikes or drops, which often mean a data-entry error.
- Alerts for low stock, upcoming expiry and forecast shortages.
- Delivered in-app and by email, with WhatsApp or SMS as an option.

Module L: AI assistant (ML)

Purpose: Answer questions in everyday language.

- Chat where owners ask, for example, "Which dish did we waste most last month?" or "How much paneer should I order for the weekend?"
- A large language model calls the app's own APIs, so answers come from real data, not guesses.

Module M: Surplus redistribution

Purpose: Turn unavoidable leftovers into donations.

- Suggests nearby NGOs or food banks when surplus is predicted or logged.
- Donation log and a "meals donated" counter on the dashboard.

Module N: Reports and administration

Purpose: Give owners records and give the platform team control.

- Weekly and monthly reports exported as PDF or Excel.
- Admin panel for tenants, usage statistics and model accuracy drift over time.

5. Machine learning components

Six ML components power the product. Demand forecasting is the core; the others add explanation, vision, monitoring and conversation.

Component	Technique	Input	Output	How it is evaluated
Demand forecasting	LightGBM / XGBoost, compared with Prophet and a naive baseline	Past sales, calendar, weather, events	Plates per dish per day	MAE, MAPE, backtesting on held-out weeks
Uncertainty	Quantile regression (10th, 50th, 90th percentile)	Same as forecasting	Low, likely, high range	Coverage: share of actuals inside the range
Explainability	SHAP values turned into sentences	Trained model + one forecast	Top reasons for the number	Manual review with users
Waste recognition	Fine-tuned CNN or YOLO detector	Photo of leftovers	Food items + estimated amount	Accuracy, precision, recall on a test set
Anomaly detection	Isolation Forest or statistical thresholds	Daily sales per dish	Flags on unusual days	Precision on injected test anomalies
Menu engineering	K-means clustering	Popularity and margin per dish	Star, plowhorse, puzzle, dog groups	Interpretability and user feedback

The AI assistant is a seventh, integration-level component: a large language model with tool calling over the app's APIs.

Forecasting features

- **Calendar:** day of week, month, weekend flag, holiday flag, days to and from the nearest holiday, payday flag.
- **Weather:** temperature, rainfall, humidity for the outlet's location.
- **Lag features:** sales of the dish 1, 7 and 14 days earlier.
- **Rolling statistics:** 7-day and 28-day mean and standard deviation.
- **Dish attributes:** category, price, promotion flag, days since the dish was launched.

Training and serving pipeline

1. Nightly job fetches tomorrow's weather and holiday data.

2. Features are built from the database and the external data.
3. The current model per outlet produces 7-day forecasts, stored in PostgreSQL.
4. Weekly job retrains on all data, compares models on the last 4 weeks, and promotes the winner.
5. Accuracy is logged so drift can be spotted on the admin panel.

Cold start

A new outlet with little history uses a global model trained across all outlets and public data. It switches to a per-outlet model once it has about 8 weeks of its own sales.

Datasets for development

The team can start before any real restaurant joins. Good candidates on Kaggle are "Food Demand Forecasting" and "Recruit Restaurant Visitor Forecasting"; Food-101 suits the vision model. These names come from memory without a live search, so verify them before use. A synthetic data generator should also be built for demos and testing.

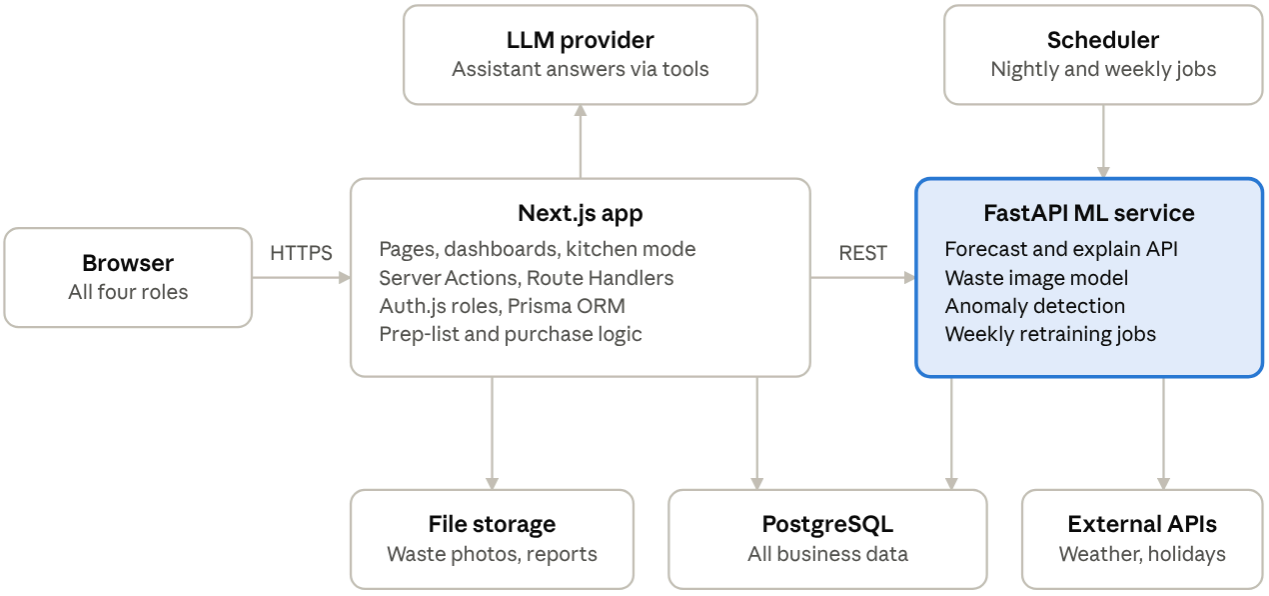
6. Tech stack and architecture

The product is a Next.js app; all machine learning lives in a separate Python FastAPI service so each side can be built and scaled independently.

Layer	Technology
Frontend and app backend	Next.js 15 (App Router), TypeScript, Tailwind CSS, shadcn/ui, Recharts
Authentication	Auth.js with role-based access
Database	PostgreSQL with Prisma ORM
ML service	Python, FastAPI, pandas, scikit-learn, LightGBM, Prophet, SHAP
Computer vision	PyTorch or Ultralytics YOLO
Background jobs	Cron jobs, or a queue such as BullMQ with Redis
File storage	S3-compatible storage, for example Supabase Storage
AI assistant	Vercel AI SDK with an LLM provider
External data	Open-Meteo for weather, a public holidays API

Layer	Technology
DevOps	Docker, GitHub Actions, Vercel (web), Render or Railway (ML)

Next.js runs the product; a Python service runs the ML



system architecture · 2 services, 6 supporting parts

The browser only talks to Next.js. Next.js calls the ML service over REST, and both read the same PostgreSQL database. The scheduler triggers nightly forecasts and weekly retraining in the ML service.

7. Database design

PostgreSQL holds 14 core tables, managed through Prisma. Every business table carries an `outletId` so tenant data stays separated.

Table	Key fields	Belongs to
Organization	name, plan	—
Outlet	name, city, latitude, longitude, timezone	Organization
User	name, email, role	Organization (+ assigned outlets)
Dish	name, category, price, isVeg, isActive	Outlet

Table	Key fields	Belongs to
Ingredient	name, unit, costPerUnit, shelfLifeDays	Outlet
RecipeItem	qtyPerPlate	Dish + Ingredient
Supplier	name, contact, leadTimeDays	Outlet
StockMovement	type (in/out), qty, expiryDate	Ingredient
SalesRecord	date, shift, qtySold, price	Dish
Forecast	date, low, likely, high, modelVersion, explanation	Dish
PrepPlan	date, status, approvedBy	Outlet (+ PrepPlanItem rows)
WasteLog	date, qty, reason, photoUrl, aiConfidence	Dish or Ingredient
PurchaseOrder	status, expectedDate	Supplier (+ items)
Alert	type, message, isRead	Outlet

Supporting tables: ExternalDay (weather and holiday per city and date), ModelRun (training metrics per outlet), and Donation (surplus given away).

8. Team division

Each member owns a slice of modules end to end, so everyone learns frontend, backend, ML and DevOps on real features. Testing, documentation and the final demo are shared.

Member	Theme	Modules owned	ML piece
Himanshu	Forecasting and planning	E, F, J	Demand forecasting with quantile ranges
Veeraj	Kitchen operations and waste vision	G, H, M	Waste image recognition
Nikhil	Data, menu and inventory	B, C, D, K	Anomaly detection on sales
Megh	Platform, insights and assistant	A, I, L, N	SHAP explanations, menu clustering, AI assistant

Himanshu: Forecasting and planning (E, F, J)

- **Frontend:** forecast view per dish with low–likely–high ranges, daily prep list page with approve and adjust, printable kitchen sheet, what-if simulator UI.
- **Backend:** Route Handlers that call the ML service, Forecast and PrepPlan tables, prep-list calculation from recipes, saving manager adjustments.
- **ML:** feature engineering, baseline vs. Prophet vs. LightGBM comparison, quantile regression, backtesting, the FastAPI forecast and what-if endpoints.
- **DevOps:** Dockerfile for the FastAPI service, nightly forecast and weekly retraining cron jobs, model versioning.

Veeraj: Kitchen operations and waste vision (G, H, M)

- **Frontend:** tablet-friendly kitchen mode, shift-wise sales logging, waste photo capture with a confirm-or-correct screen, closing checklist, donation suggestions page.
- **Backend:** SalesRecord, WasteLog and Donation APIs, photo upload to file storage, day-locking logic at closing.
- **ML:** collect and label food images, fine-tune a CNN or YOLO model, evaluate accuracy, the FastAPI image recognition endpoint.
- **DevOps:** file storage bucket and access rules, image model serving and size optimisation, upload limits and compression.

Nikhil: Data, menu and inventory (B, C, D, K)

- **Frontend:** CSV upload with column-mapping wizard, menu and recipe builder, inventory and supplier pages, purchase order screens, alerts panel.
- **Backend:** CSV parsing and validation, Dish, Ingredient, RecipeItem, StockMovement and PurchaseOrder APIs, cost-per-plate logic, alert rules for stock and expiry.
- **ML:** anomaly detection (Isolation Forest vs. statistical thresholds) to flag unusual sales days; also builds the synthetic data generator the whole team uses.
- **DevOps:** Prisma migrations and seed scripts, managed PostgreSQL setup, email service for alerts.

Megh: Platform, insights and assistant (A, I, L, N)

- **Frontend:** login, signup and invitation pages, role-aware navigation, analytics dashboard and waste heatmap, AI assistant chat, admin panel.
- **Backend:** Auth.js with roles and multi-tenancy, permission checks on every action, report generation (PDF and Excel), the assistant's tool definitions.
- **ML:** SHAP explanations turned into plain sentences, K-means menu engineering, LLM tool calling over the app's APIs.

- **DevOps:** GitHub Actions CI (lint, type-check, tests), Vercel and Render or Railway deployments, environment variables and secrets, error monitoring.

Shared foundations (weeks 1–2, done together)

- Prisma schema and the API contract between Next.js and FastAPI.
- Design system: colours, components, layout shell.
- Repository structure, coding conventions and Docker Compose for local development.

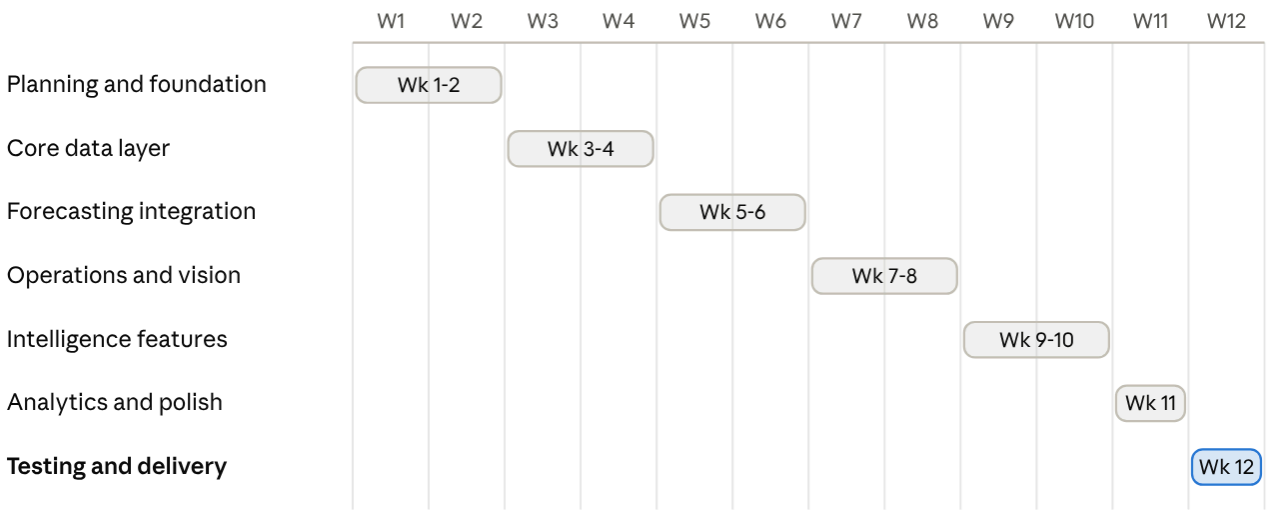
Megh's SHAP work depends on Himanshu's forecasting model, so those two should agree the model's input and output format by week 4.

Ways of working

- One GitHub repository with `web/` (Next.js) and `ml/` (FastAPI) folders.
- Feature branches with pull requests; each PR is reviewed by someone from a different slice, so knowledge spreads across the team.
- Contracts first: write the API and database shape before building, and use mock responses so no one is blocked.
- A short weekly demo where each member shows one frontend, backend, ML or DevOps change and explains how it works.
- A weekly 30-minute sync and a shared task board.

9. Timeline, scope and success

The project runs for 12 weeks in seven phases, with a working MVP by the end of week 8.



project timeline · 7 phases over 12 weeks

What each phase delivers

1. **Planning and foundation (weeks 1–2):** requirements, wireframes, database schema, API contract, repository and CI setup, dataset exploration.
2. **Core data layer (weeks 3–4):** auth and roles, menu and recipes, CSV import, inventory basics, baseline and first LightGBM model.
3. **Forecasting integration (weeks 5–6):** FastAPI forecasting connected to Next.js, prep lists, weather and holiday data, nightly job.
4. **Operations and vision (weeks 7–8):** kitchen mode, daily logging, waste tracking with the image model, purchase orders, alerts.
5. **Intelligence features (weeks 9–10):** SHAP explanations, what-if simulator, anomaly detection, menu engineering, AI assistant.
6. **Analytics and polish (week 11):** dashboards, reports, surplus donation, admin panel, performance and UI polish.
7. **Testing and delivery (week 12):** end-to-end testing, deployment, documentation, final report, demo video.

MVP and stretch goals

MVP (must have): auth with roles, menu and recipes, sales import, forecasting with baseline comparison, prep list, daily logging, manual waste tracking, basic dashboard.

Stretch goals: waste image recognition, AI assistant, what-if simulator, POS integration, WhatsApp alerts, surplus donation, multi-outlet comparison.

Success metrics

- Forecast MAPE clearly lower than the "same day last week" baseline on held-out weeks.
- At least 80% of actual sales fall inside the predicted low–high range.
- Waste image model accuracy reported on a held-out test set.
- Measured or simulated waste reduction against historical data.
- Ideally, a 2–4 week pilot with a real café or college canteen, with before and after waste figures.

Deliverables

- Deployed web application and ML service.
- GitHub repository with a clear README and setup guide.
- Jupyter notebooks showing data analysis and model comparison.
- Project report with architecture, ML evaluation and results.
- Demo video covering the full flow from data upload to waste reduction.

Risks and mitigations

Risk	Mitigation
No real restaurant data early on	Start with public datasets and a synthetic data generator
Image model not accurate enough	Keep manual correction; treat vision as a stretch goal
ML and frontend blocked on each other	Agree the API contract in week 1; use mock responses
Scope too large for 12 weeks	Lock the MVP first; stretch goals only after week 8
Forecasts distrusted by users	Show ranges and plain-language explanations with every number